

ASP.NET Core Authentication Strategies

ASP.NET Core provides an extensible "Authentication Scheme" system. You choose the strategy based on the *type* of application you are building. Here are the 4 primary authentication strategies used in modern .NET architecture:

1. Cookie Authentication (The Traditional Web App Standard)

- **What it is:** When a user logs in (e.g., enters a username/password), the server creates an encrypted cookie containing their digital passport and sends it to the browser. The browser automatically attaches this cookie to every future HTTP request.
- **When to use it:** When you are building Server-Side Rendered (SSR) applications like **MVC (Controllers + Views)**, **Razor Pages**, or **Blazor Server**.
- **Why we use it:** Browsers handle cookies automatically, making it incredibly secure and easy to use for traditional websites.

2. JWT Bearer Authentication (The Modern API Standard)

- **What it is:** JWT stands for **JSON Web Token**. Instead of a cookie, the server gives the client a long, encrypted string (the token). The client (like a mobile app or a React frontend) must manually attach this token to the `Authorization: Bearer <token>` header of every single HTTP request it sends to your Minimal API.
- **When to use it:** When you are building a pure backend **Minimal API** or REST API that is consumed by **Single Page Applications (React, Angular, Vue)** or **Mobile Apps (iOS, Android)**.
- **Why we use it:** Mobile apps and SPAs don't handle cookies as seamlessly as traditional web pages. JWTs are stateless, meaning the server doesn't need to look up a session in the database for every request—it just cryptographically verifies the token's signature.

3. ASP.NET Core Identity (The Complete Ecosystem)

- **What it is:** Microsoft's built-in, massive membership system. It is not just a token handler; it is a complete UI and database solution. It generates the SQL tables for Users, Roles, and Passwords via Entity Framework Core. It also provides pre-built logic for Two-Factor Authentication (2FA), Password Resets, and Email Confirmation.
- **When to use it:** When you need a full user-management database (where you store user emails, hashed passwords, etc.) and you don't want to build the "Forgot Password" or "Login" logic from scratch.

***Note:** ASP.NET Core Identity internally uses Cookie Authentication for its web views, but in .NET 8, they added `MapIdentityApi()` which allows it to easily issue JWT Bearer tokens for Minimal APIs too!*

4. OpenID Connect (OIDC) / OAuth 2.0 (The Outsourcer)

- **What it is:** Delegated authentication. Instead of forcing the user to create a new password for your app, you redirect them to a trusted third party (like Google, Microsoft Entra ID, or Auth0). The third party verifies who the user is and sends a token back to your application saying, "We verified this is John Doe."
- **When to use it:** When you want to implement "**Log in with Google**" or "**Log in with Apple**", or when you are building Enterprise software that needs to integrate with a company's existing Active Directory (SSO / Single Sign-On).
- **Why we use it:** You don't have to worry about storing user passwords in your database. You let a multi-billion dollar tech company handle the massive security risk of password management.

Decision Matrix Cheat Sheet

```
/*
 * =====
 * ASP.NET CORE AUTHENTICATION DECISION MATRIX
 * =====
 *
 * SCENARIO 1: You are building a Web API for a React/Angular/Mobile App.
 * -> STRATEGY: JWT Bearer Authentication (Tokens).
 *
 * SCENARIO 2: You are building an MVC, Razor Pages, or Blazor web app.
 * -> STRATEGY: Cookie Authentication.
 *
 * SCENARIO 3: You need a database to store users, passwords, and roles now.
 * -> STRATEGY: ASP.NET Core Identity.
 *
 * SCENARIO 4: You want "Log in with Google" or Corporate Single Sign-On (SSO).
 * -> STRATEGY: OpenID Connect (OIDC) / OAuth 2.0.
 *
 * =====
 */
```